

Nama : Yudhitira Putra Hartanto

Kelas/absen : 1G/29

Prodi : D4 Teknik Informatika

Praktikum Menghitung Nilai Faktorial dengan Algoritma Brute Force dan Divide and Conquer

1. Class Faktorial

```
package BruteForceDivideConquer.minggu5;
```

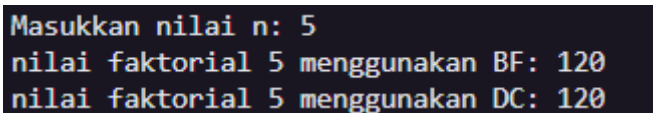
```
public class Faktorial {  
    int faktorialBF(int n){  
        int fakto = 1;  
        for(int i=1; i<=n; i++){  
            fakto= fakto * i;  
        }  
        return fakto;  
    }  
    int faktorialDC(int n){  
        if(n==1){  
            return 1;  
        }else{  
            int fakto = n * faktorialDC(n-1);  
            return fakto;  
        }  
    }  
}
```

2. Class MainFaktorial

```
package BruteForceDivideConquer.minggu5;
```

```
import java.util.Scanner;
```

```
public class MainFaktorial {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Masukkan nilai n: ");  
        int nilai = sc.nextInt();  
  
        Faktorial f = new Faktorial();  
        System.out.println("nilai faktorial "+nilai+" menggunakan BF: " +  
f.faktorialBF(nilai));  
        System.out.println("nilai faktorial "+nilai+" menggunakan DC: " +  
f.faktorialDC(nilai));  
    }  
}
```

3. Hasil : 

Soal

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!
2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!
3. Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1); !
4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!

Jawaban

1. Penggunaan if pada if(n==1) merupakan kondisi dasar, yang ketika nilai n mencapai, rekursi dihentikan dengan mengembalikan nilai 1. Bagian else merupakan langkah rekursif yang memecahkan masalah faktorial menjadi masalah yang lebih kecil yaitu (n-1), kemudian hasilnya dikombinasikan dengan mengalikannya dengan n.
2. Memungkinkan dengan mengganti perulangan dengan jenis perulangan lain seperti while :

```
package BruteForceDivideConquer.minggu5;
```

```
public class Faktorial {  
    int faktorialBF(int n){  
        int fakto = 1;  
        int i = 1;  
        while (i <= n) {  
            fakto *= i;  
            i++;  
        }  
        return fakto;  
    }  
    int faktorialDC(int n){  
        if(n==1){  
            return 1;  
        }else{  
            int fakto = n * faktorialDC(n-1);  
            return fakto;  
        }  
    }  
}
```

3. Iteratif (*=) : perhitungan dilakukan dalam satu lingkup fungsi dengan pengulangan.
Rekursif (n * faktorialDC(n-1)) : perhitungan dilakukan dengan memanggil diri sendiri berulang, memanfaatkan stack.
4. faktorialBF() menggunakan pendekatan iteratif (perulangan for), menghitung faktorial dengan mengalikan bilangan dari 1 hingga n secara langsung.
faktorialDC() menggunakan pendekatan rekursif (divide and conquer), memecah masalah dengan memanggil dirinya sendiri untuk n-1 hingga mencapai base case (n==1), lalu menggabungkan hasil perkalian. Iteratif lebih hemat memori dan cepat, rekursif lebih elegan namun boros stack untuk n besar.

Menghitung Hasil Pangkat dengan Algoritma Brute Force dan Divide and Conquer

1. Class Pangkat

```
package BruteForceDivideConquer.minggu5;

public class Pangkat {
    int nilai, pangkat;

    Pangkat(int n, int p){
        nilai = n;
        pangkat = p;
    }

    int pangkatBF(int a, int n){
        int hasil = 1;
        for(int i=0; i<n; i++){
            hasil = hasil*a;
        }
        return hasil;
    }

    int pangkatDC(int a, int n){
        if(n==1){
            return a;
        }else{
            if(n%2==1){
                return (pangkatDC(a, n/2)*pangkatDC(a, n/2)*a);
            }else{
                return (pangkatDC(a, n/2)*pangkatDC(a, n/2));
            }
        }
    }
}
```

2. Class MainPangkat

```
package BruteForceDivideConquer.minggu5;

import java.util.Scanner;

public class MainPangkat {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();

        Pangkat[] png = new Pangkat[elemen];
        for(int i=0; i<elemen; i++){
            System.out.println("Masukkan nilai basis elemen ke- "+(i+1)+" : ");
            int basis = input.nextInt();
            System.out.println("Masukkan nilai pangkat elemen ke- "+(i+1)+" : ");
            int pangkat = input.nextInt();
            png[i] = new Pangkat(basis, pangkat);
        }
    }
}
```

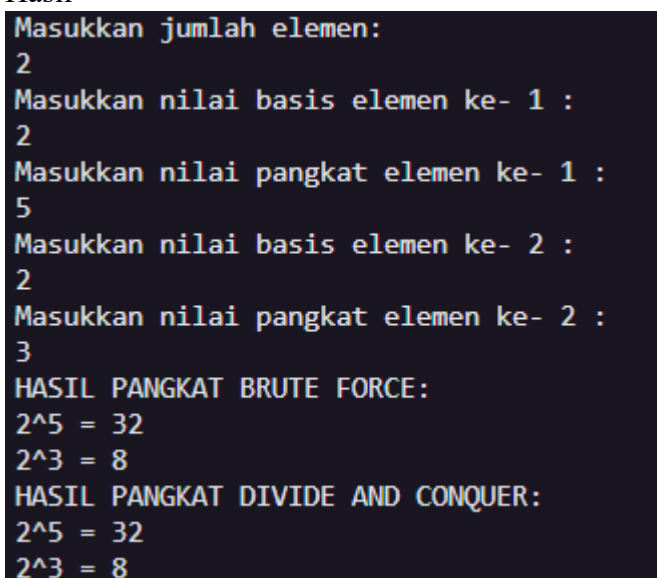
```

    }

    System.out.println("HASIL PANGKAT BRUTE FORCE: ");
    for(Pangkat p : png){
        System.out.println(p.nilai+" pangkat "+p.pangkat+" = "+p.pangkatBF(p.nilai,
p.pangkat));
    }
    System.out.println("HASIL PANGKAT DIVIDE AND CONQUER: ");
    for(Pangkat p : png){
        System.out.println(p.nilai+" pangkat "+p.pangkat+" = "+p.pangkatDC(p.nilai,
p.pangkat));
    }
}
}
}

```

3. Hasil



```

Masukkan jumlah elemen:
2
Masukkan nilai basis elemen ke- 1 :
2
Masukkan nilai pangkat elemen ke- 1 :
5
Masukkan nilai basis elemen ke- 2 :
2
Masukkan nilai pangkat elemen ke- 2 :
3
HASIL PANGKAT BRUTE FORCE:
2^5 = 32
2^3 = 8
HASIL PANGKAT DIVIDE AND CONQUER:
2^5 = 32
2^3 = 8

```

Soal

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!
2. Apakah tahap combine sudah termasuk dalam kode tersebut? Tunjukkan!
3. Pada method pangkatBF() terdapat parameter untuk melewati nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?
4. Tarik tentang cara kerja method pangkatBF() dan pangkatDC()!

Jawaban

1. pangkatBF(): iteratif, menggunakan perulangan for sebanyak n kali.
pangkatDC(): rekursif (divide and conquer), memecah n menjadi dua bagian, lalu menggabungkan hasil.
2. Iya termasuk karena tahap combine terdapat pada bagian return yang mengalikan hasil-hasil sub-masalah:
 - Genap: $\text{pangkatDC}(a, n/2) * \text{pangkatDC}(a, n/2)$
 - Ganjil: ditambah $* a$
3. Tetap relevan dengan parameter agar lebih fleksibel. Bisa, dengan memanfaatkan atribut nilai dan pangkat. Contoh:

- ```

int pangkatBF() {
 int hasil = 1;
 for (int i = 0; i < this.pangkat; i++) {
 hasil *= this.nilai;
 }
 return hasil;
}

```
4. pangkatBF(): hasil dikalikan basis secara berulang.  
 pangkatDC(): rekursif dengan aturan:
- Jika  $n=1 \rightarrow \text{return } a$
  - Jika  $n$  genap  $\rightarrow \text{return } (a^{(n/2)})^2$
  - Jika  $n$  ganjil  $\rightarrow \text{return } (a^{(n/2)})^2 * a$

## Menghitung Sum Array dengan Algoritma Brute Force dan Divide and Conquer

1. Class Sum
 

```
package BruteForceDivideConquer.minggu5;
```

```

public class Sum {
 double keuntungan[];
 Sum(int el){
 keuntungan = new double[el];
 }

 double totalBF(){
 double total=0;
 for(int i=0;i<keuntungan.length;i++){
 total += keuntungan[i];
 }
 return total;
 }

 double totalDC(double arr[], int l, int r){
 if(l==r){
 return arr[l];
 }else{
 int mid = (l+r)/2;
 double lsum = totalDC(arr, l, mid);
 double rsum = totalDC(arr, mid+1, r);
 return lsum + rsum;
 }
 }
}

```

2. Class MainSum
 

```
package BruteForceDivideConquer.minggu5;
```

```

import java.util.Scanner;

public class MainSum {
 public static void main(String[] args) {

```

```

Scanner input = new Scanner(System.in);
System.out.print("Masukkan jumlah elemen: ");
int elemen = input.nextInt();

Sum sm = new Sum(elemen);
for(int i=0; i<elemen; i++) {
 System.out.print("Masukkan keuntungan ke-" + (i+1) + ": ");
 sm.keuntungan[i] = input.nextDouble();
}

System.out.println("Total keuntungan menggunakan Bruteforce: " +
sm.totalBF());
 System.out.println("Total keuntungan menggunakan Divide and Conquer: " +
sm.totalDC(sm.keuntungan, 0, elemen-1));
}

}

```

### 3. Hasil

```

Masukkan jumlah elemen: 5
Masukkan keuntungan ke-1: 10
Masukkan keuntungan ke-2: 20
Masukkan keuntungan ke-3: 30
Masukkan keuntungan ke-4: 40
Masukkan keuntungan ke-5: 50
Total keuntungan menggunakan Bruteforce: 150.0
Total keuntungan menggunakan Divide and Conquer: 150.0

```

### Soal

1. Kenapa dibutuhkan variable mid pada method TotalDC()?
2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

```
double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);
```
3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

```
return lsum+rsum;
```
4. Apakah base case dari totalDC()?
5. Tarik Kesimpulan tentang cara kerja totalDC()

### Jawaban

1. Variabel mid digunakan untuk menentukan titik tengah array, sehingga array dapat dibagi menjadi dua bagian (kiri dan kanan) yang akan diproses secara rekursif.
2. double lsum = totalDC(arr, l, mid); dan double rsum = totalDC(arr, mid+1, r); berfungsi untuk menghitung total keuntungan dari bagian kiri dan kanan secara terpisah melalui rekursi (tahap divide).
3. Penjumlahan lsum + rsum adalah tahap *combine* untuk menggabungkan hasil dari kedua bagian sehingga diperoleh total keseluruhan array.
4. Base case terjadi ketika  $l == r$ , yaitu hanya tersisa satu elemen, maka method mengembalikan nilai arr[l].
5. Method totalDC() menerapkan algoritma Divide and Conquer: memecah array menjadi dua bagian secara rekursif hingga elemen tunggal, lalu menjumlahkan kembali hasil dari setiap bagian untuk mendapatkan total keseluruhan.